

Q. No. 1: What will be the output of the first three phases of compiler when the following assignment statement passes through them.

$\text{area} = \text{length} + 2.5 * (\text{width} * \text{height})$

(All variables are of type 'real')

Solution:



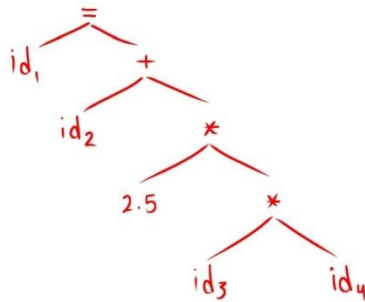
Lexical Analyzer



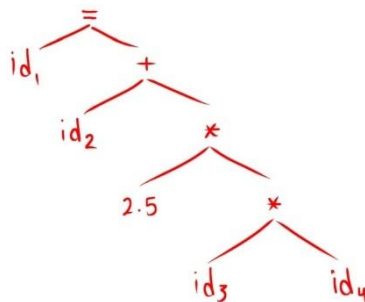
$\langle id_1 \rangle \langle = \rangle \langle id_2 \rangle \langle + \rangle \langle 2.5 \rangle \langle * \rangle \langle (\rangle \langle id_3 \rangle \langle * \rangle \langle id_4 \rangle \langle) \rangle$



Syntax Analyzer



Semantic Analyzer



Q. No. 2: MCQs.

1) What of the following is **NOT TRUE** about Lexical Analysis?

- a) Return the next token to the scanner.
- b) Ignores the blanks and comments.
- c) Groups the characters into lexemes
- d) Produce token stream.

2) Processes the source code only **once** and produces object code.

- a) one pass compilers.
- b) multi pass compilers.
- c) Incremental Compilers.
- d) Cross Compilers.

3) What of the following is **NOT TRUE** about Symbol Table?

- a) Allow all front-end and back-end phases to find the records.
- b) The lexical analyzer inserts identifiers into the symbol table when detected.
- c) Contains information about the type, scope, or storage of identifiers.
- d) Each compiler phase creates its own separate symbol table unrelated to others.

4) The **assembler** converts the assembly code into...

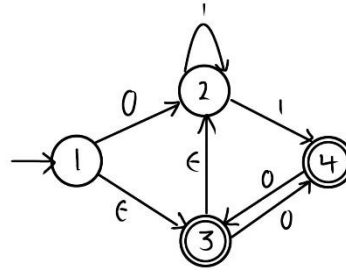
- a) Intermediate code.
- b) Relocatable machine code.
- c) High-level source code.
- d) Absolute machine code.

5) How many tokens will be created by the lexical analyzer for the following statement?

While (10 > 1 && 7 = 5)

- a) 8
- b) 10
- c) 11
- d) 12

Q. No. 3: Convert the following NFA into its equivalent DFA, by showing all steps in detail. Without detailed calculations, you will get 0 marks.



a) Calculations:

$$\epsilon\text{-closure}(1) = \{1, 2, 3\} = A$$

$$\epsilon\text{-closure}(\text{move}(A,0)) = \epsilon\text{-closure}(2, 4) = \{2, 4\} = B$$

$$\epsilon\text{-closure}(\text{move}(A,1)) = \epsilon\text{-closure}(2, 4) = B$$

$$\epsilon\text{-closure}(\text{move}(B,0)) = \epsilon\text{-closure}(3) = \{2, 3\} = C$$

$$\epsilon\text{-closure}(\text{move}(B,1)) = \epsilon\text{-closure}(2, 4) = B$$

$$\epsilon\text{-closure}(\text{move}(C,0)) = \epsilon\text{-closure}(4) = \{4\} = D$$

$$\epsilon\text{-closure}(\text{move}(C,1)) = \epsilon\text{-closure}(2, 4) = B$$

$$\epsilon\text{-closure}(\text{move}(D,0)) = \epsilon\text{-closure}(3) = C$$

$$\epsilon\text{-closure}(\text{move}(D,1)) = \epsilon\text{-closure}(\emptyset) = \{\emptyset\} = E$$

$$\epsilon\text{-closure}(\text{move}(E,0)) = \epsilon\text{-closure}(\emptyset) = E$$

$$\epsilon\text{-closure}(\text{move}(E,1)) = \epsilon\text{-closure}(\emptyset) = E$$

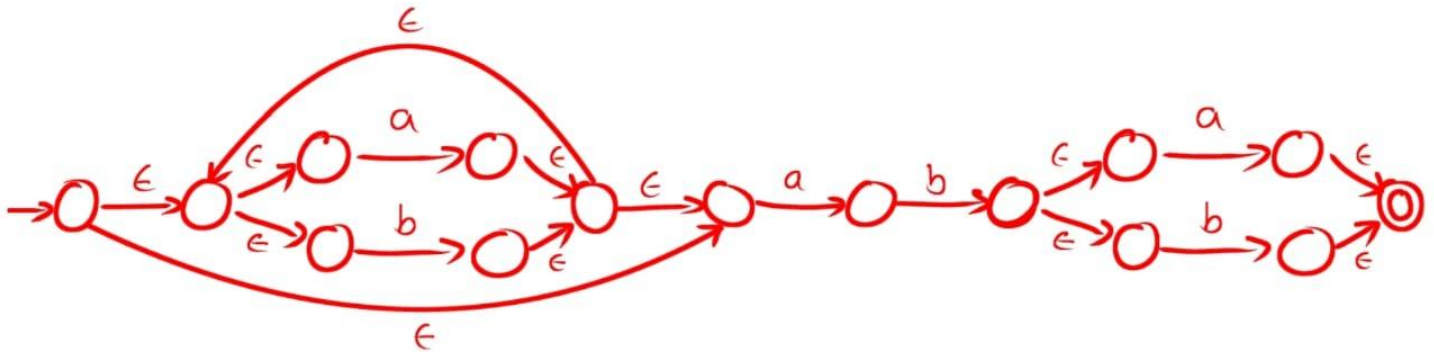
b) State Table for DFA:

	0	1
A	B	B
B	C	B
C	D	B
D	C	E
E	E	E

Q. No. 4: Solve these NFA questions:

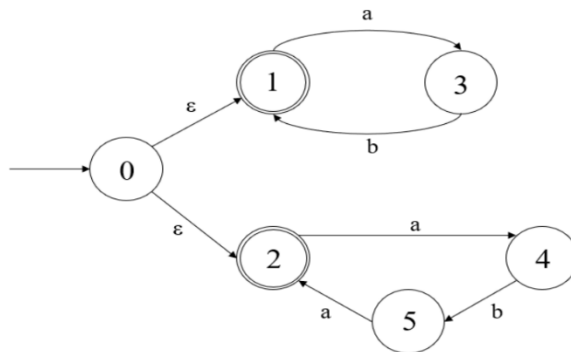
a) Draw the NFA for the following regular expression: $(a|b)^* ab(a|b)$

(The doctor said at least write 4 steps, not directly the final solution. Here I only solved the final one to save time)



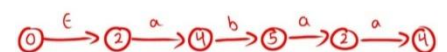
b) Check if the strings are accepted in the NFA below and draw their path:

(I couldn't remember the exact NFA but the idea is the same)



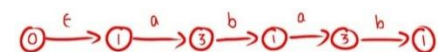
1- The pattern abaa is (Accepted/Rejected) using the above NFA.

Draw the path:



2- The pattern abab is (Accepted/Rejected) using the above NFA.

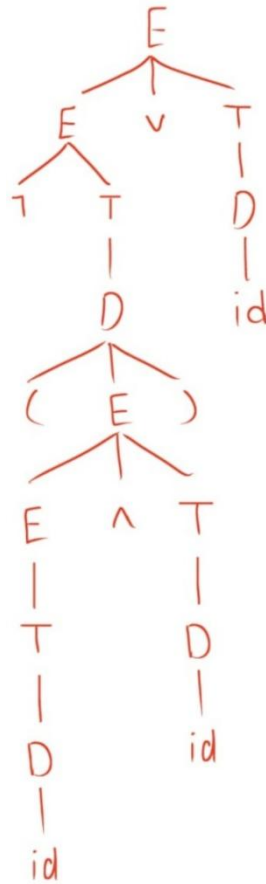
Draw the path:



Q. No. 5: Consider the following grammar.

$$\begin{aligned} E &\rightarrow E \vee T \mid E \wedge T \mid T \\ T &\rightarrow \neg T \mid D \\ D &\rightarrow (E) \mid \text{id} \end{aligned}$$

a) Construct a parse tree for the expression $\neg(\text{id} \wedge \text{id}) \vee \text{id}$



b) Is this grammar ambiguous? explain why.

Yes, the grammar is ambiguous.

$E \wedge T$ has higher precedence than $E \vee T$, so they should be in separate productions.

Q. No. 6: Consider the following grammar.

$$S \rightarrow \text{if } B T ;$$
$$B \rightarrow \text{true} \mid \text{false}$$
$$T \rightarrow \text{id} = \text{val}$$
$$\text{id} \rightarrow a \mid b$$
$$\text{val} \rightarrow 0 \mid 1 \mid 2$$

Use **right** derivation to produce this output: **if false a = 2;**

$$\begin{aligned} S &\rightarrow \text{if } B T ; \\ &\rightarrow \text{if } B \text{ id} = \text{val} ; \\ &\rightarrow \text{if } B \text{ id} = 2 ; \\ &\rightarrow \text{if } B a = 2 ; \\ &\rightarrow \text{if false } a = 2 ; \end{aligned}$$

Q. No. 7: Given this sample code:

```
int total = 0 ;  
While ( 100 > total ) {  
    total = total + 10 ;  
}
```

Fill this table:

Lexemes	Token_names	Does it enter the symbol table (Yes/No)
int	keyword	No
total	id	Yes
=	equal sign	No
0	number	Yes
;	semicolon	No
While	keyword	No
(	left parenthesis	No
100	number	Yes
>	relop	No
total	id	Yes
)	right parenthesis	No
{	left curly bracket	No
total	id	Yes
=	equal sign	No
total	id	Yes
+	add	No
10	number	Yes
;	semicolon	No
}	right curly bracket	No